

Compiling Optimal Retry Policies with Charged Atomic Calls

ANONYMOUS AUTHOR(S)

We synthesize retry policies for dependent transformations using three established atomic-call modes: cheap validation, fresh protected computation, and cached reuse or recomputation. A potential proves optimality over programs retaining multiple preparations; identity-based Java callbacks admit a matching adversary with at most B actual writes. The resource frontier is exact: one policy, without a writer-budget or price input, permits r extra calls and simultaneously minimizes worst protected work for every B , to the sum of the largest $\min((B - r)_+, n)$ job works. Without cached completion, all work must be protected already at $B = r$. Among fixed-order versions of this policy, minimum-work ready ordering also minimizes worst total work for every B, r . All-budget scalar values have two input-derived slopes per job, at most $4n$ positive marginal runs per state, and $2^n \text{poly}(L)$ construction/checking for binary input length L . A mixed-price class admits $O(|E| + n \log(n + 1))$ construction/checking; a zero-fee subcase is NP-complete at $B = 1$. For arbitrary independent charged jobs, the two-/three-mode total-cost ratio is at most $(1 + \sqrt{2})/2$, sharply. Direct equations, native resource counters and certificates test these results. A Deephaven integration preserves captured directives under caller-owned refresh bounds. API calibration fails to establish measured third-mode benefit. The guarantees concern counted resources; application prevalence and elapsed-time improvement remain unestablished.

CCS Concepts: • **Software and its engineering** → **Formal software verification**; *Concurrent programming structures*.

Additional Key Words and Phrases: program synthesis, concurrency, retries, quantitative games, resource guarantees

1 Introduction

Preparing and validating a shared-object transformation risks repeated work; atomic computation pays for protection. Dependencies require choosing the next ready transformation. Optimal retry synthesis must account for every available operation.

Let W count all kernel work, L the part inside callbacks, and Q completion/validation calls. For dependent kernels of 8 and 16 selected operations, preparing outside and cached-completing gives $Q = 2$ without knowing the writer bound. Worst protected work is 0, 16, or 24 for at most 0, 1, or 2 writes. Every two-mode program with the same universal call cap must protect all 24 operations even in a quiet execution (Theorem 4). Cached completion changes joint resource feasibility without a fitted protection price or a caller-certified interference limit.

Table 1. Constructed two-job native comparison at $B = 1$. Columns give separate worst values over each policy's complete branch set. The maximum weighted cost $R = W + L + 16Q$ is computed per execution; it is not the sum of component maxima.

Policy	$\max W$	$\max L$	$\max Q$	$\max R$
Optimal three-mode	32	16	2	72
Optimal two-mode, protection-first ties	24	24	2	80
Optimal two-mode, cheap-first ties	32	16	3	80

Table 1 optimizes the scalar metric with known $B = 1$, $p_i = w_i$, $v_i = k_i = 16$. Cached recomputation adds work but shares the completing call. Validation-only callbacks must also be priced: they can cheaply reject a mismatch.

The interface permits retained preparations, discards and ready-job switching, but requires whole kernels, captured predecessors, persistent completion and call-scoped protection. Its normal form

yields a game on unfinished sets and remaining interference. Two slopes per job represent entire value curves; a checker verifies their Bellman equations on finite affine partitions. Mixed prices admit $O(|E| + n \log(n + 1))$ construction/checking with linear cursor space. General compilation is NP-complete even at one failure in a zero-fee subcase; independent jobs have sharp two-/three-mode total-cost ratio $(1 + \sqrt{2})/2$.

Java identity callbacks realize the game with at most B writes, including colliding keys. Deephaven source bounds and caller-owned refreshes provide only an upper guarantee: extra phase information improves its game value.

Budgeted DP, reflected recurrences and input-slope envelopes are established [8, 27, 31]. The contribution is their justified specialization to charged atomic calls: the normal-form potential, simultaneous resource frontier without a writer-budget input, finite slope-basis closure, sharp independent-job ratio, and exact charged-price reduction that removes both budget and subset enumeration on a stated class. Authored experiments test these claims and their implementation.

2 Charged Atomic Calls

Write $\mathbb{N} = \{0, 1, \dots\}$. Jobs $J = \{1, \dots, n\}$ form a finite DAG with predecessor set $\text{Pred}(i)$. Implementation IDs start at zero; readiness requires all predecessors completed. Completion records the exact immutable output and persists despite later external writes. The objective is one successful foreground transformation per job, not a simultaneous snapshot or transaction.

Each ready job has a pure, opaque, whole kernel receiving its own-input identity and exact completed predecessor outputs. Its fixed unprotected work is $w_i > 0$; under protection it costs $w_i + p_i$, $p_i \geq 0$, including predecessor processing. Kernels finish with exact results; partial/incremental evaluation, preparation before readiness and discharge by external work are excluded. A preparation is a capability tied to its input identity and predecessor tuple. Captured outputs cannot be reconstructed by later reads of the mutable map.

A local state is U (no usable current preparation), C (a usable current preparation), or D (completed). Several ready jobs may simultaneously be in C. Initially every job is U. An invalidation changes an own-input identity, preserving captured predecessor outputs. The environment has a global remaining allowance b , initially $B \in \mathbb{N}$. At zero allowance only matching outcomes are legal.

The declared charge $v_i \geq 0$ covers an identity-conditional publication call. The charge $k_i \geq 0$ covers entry and exit of an atomic callback, including a callback that only validates. It may be supplied as $g_i + r_i$. These charges define a resource metric; they are not measurements of API latency. All prices are integers, or rational prices scaled uniformly to integers. Table 2 defines the available counted operations. A callback holds protection only during that operation and performs no other job's foreground work.

Table 2. Counted atomic-call interface. A mismatch spends one invalidation token. Cached completion always finishes the job; validation-only mismatch retains the current live entry and reports failure.

Operation	Start	Matching outcome/cost	Mismatching outcome/cost
Prepare	U	C, w_i	—
Conditional publication	C	D, v_i	U, v_i
Validation-only callback	C	D, k_i	U, k_i
Fresh callback	U	D, $k_i + w_i + p_i$	—
Cached callback	C	D, k_i	D, $k_i + w_i + p_i$

Discarding a preparation is free. A free inspection is permitted only with a legal outcome that preserves a currently usable preparation; it cannot certify that future interference is exhausted. Programs can select any ready job after any completed call, retain preparations or repeat inspections. There is no bare acquire/release operation, guard-retaining return or cross-job batch callback. All calls finish, and every noncompleting program, including one executing free cycles forever, has infinite objective. The environment resolves outcomes after observing the program's action.

Exposed, retained guards permit choices unavailable inside this call interface. Positive guard charges defeat some serial-macro arguments; the distinct zero-fee result and charged counterexamples remain in Section 10.

3 A Three-Mode Normal Form

Write S for the unfinished jobs and

$$\mathcal{A}(S) = \{i \in S : \text{Pred}(i) \cap S = \emptyset\}$$

for available jobs. Reachable unfinished sets are upsets. For proofs we extend the game to all subsets, treating absent predecessors as completed. Define

$$\begin{aligned} m_i &= \min(v_i, k_i), & c_i &= w_i + m_i, \\ \delta_i &= k_i - m_i, & P_i &= p_i + \delta_i, \\ s_i &= w_i + p_i. \end{aligned} \tag{1}$$

An unavoidable baseline from an all-unprepared start is $A(S) = \sum_{i \in S} c_i$. Three serial modes remain at boundaries where unfinished jobs have no retained preparation:

- (1) *Protected*: execute a fresh callback, paying $c_i + P_i$, and remove i .
- (2) *Cheap validation*: prepare and invoke the cheaper validation API. A match pays c_i and removes i ; a mismatch pays c_i , leaves i unprepared and spends one token.
- (3) *Cached completion*: prepare and invoke a cached callback. A match pays $c_i + \delta_i$; a mismatch pays $c_i + \delta_i + s_i$ and spends one token. Both remove i .

Subtract the once-paid baseline. The optimal worst excess of these modes satisfies $F(\emptyset, b) = F(S, 0) = 0$ and, for $b > 0$,

$$F(S, b) = \min_{i \in \mathcal{A}(S)} \left\{ \begin{aligned} &P_i + F(S - i, b), \\ &\max\{F(S - i, b), c_i + F(S, b - 1)\}, \\ &\delta_i + \max\{F(S - i, b), s_i + F(S - i, b - 1)\} \end{aligned} \right\}, \tag{2}$$

where $S - i = S \setminus \{i\}$. At $b = 0$ cheap validation attains zero excess; the cached option has excess δ_i , without a mismatch branch. Each mode outcome decreases $|S| + b$, so a minimizing deterministic controller finishes within $n + B$ mode invocations. This is worst-path control against an adaptive environment, not expected-cost control against a fixed distribution.

THEOREM 1 (CHARGED ATOMIC-CALL NORMAL FORM). *For the interface of Section 2, the minimum worst cost over all causal programs from the all-unprepared state is*

$$\text{OPT}(J, B) = \sum_{i \in J} c_i + F(J, B).$$

A serial three-mode controller attains it. Programs retaining several preparations or changing jobs between calls cannot improve the value.

PROOF. The three modes implement Table 2 with exactly Equation (2)'s full costs. Their decreasing rank and captured successful outputs give the upper bound.

For the lower bound, let x be any interface state with unfinished set T . Define

$$\Phi(x, b) = \sum_{i:x_i=U} c_i + \sum_{i:x_i=C} m_i + F(T, b). \quad (3)$$

Cached jobs remain ready because completion persists. For each operation we show a legal outcome preserving immediate cost plus successor potential. Preparation pays $w_i = c_i - m_i$ and replaces c_i by m_i . Validation with charge $z_i \in \{v_i, k_i\}$ gives excess $z_i - m_i + F(T - i, b)$ on success or $w_i + z_i + F(T, b - 1)$ on failure. Since $z_i \geq m_i$, their maximum dominates the cheap branch; at $b = 0$ success suffices. Fresh callbacks use the protected inequality. Cached callbacks give excesses $\delta_i + F(T - i, b)$ and $\delta_i + s_i + F(T - i, b - 1)$, exactly the cached branch. Discarding C raises potential by w_i ; inspection admits a preserving outcome.

Nature selects a witnessing branch after each action, within budget b . Telescoping gives the initial potential as a cost lower bound, covering arbitrary job switching and retained preparations. Noncompletion costs infinity; intermediate potentials need not equal intermediate optima. $\square$

A resource cap M is feasible exactly when $A(J) + F(J, B) \leq M$. Prices and allowance are supplied; restricted or correlated outcomes preserve the upper guarantee but may lower the optimum.

4 Actual External Writes in Java

Repeated stale rejection need not require a new write; writes to completed parents preserve captured outputs.

Use Java 17's `ConcurrentHashMap` with one foreground thread, distinct persistent keys, immutable identity-equal wrappers and fresh, never-reused identities for external writes. Each job updates only its fixed key. Keys are not removed; results are non-null; kernels use own snapshots and captured parents and terminate without exceptions. Conditional replacement accepts identity or equals equality [24, 25]; atomic compute invokes its callback once, under a bin monitor in the inspected source [23, 25]. Different keys may share bins.

Implement Table 2 using `replace(key, expected, result)` and `compute`. Validation-only callbacks return the prepared result on match, otherwise the unchanged entry with a failure flag; cached callbacks recompute on mismatch, and fresh callbacks always compute under protection. Capture permanent milestones locally rather than rereading the map.

Callbacks perform no other-key map operations, other foreground kernels or direct guard work, including on cached matches; no guard survives. Multiple preparations are allowed. Prices charge the specified calls; bin collisions add no unperformed kernel work. Assume scheduling and call completion: finite write counts do not imply fairness.

THEOREM 2 (MATCHING ACTUAL-WRITE ENVIRONMENT). *Under these conditions, with no initial preparations and at most B successful external writes,*

$$\min_P \sup_{E:\#writs(E) \leq B} \text{Cost}(P, E) = \sum_i c_i + F(J, B).$$

The equality holds for separate or shared internal bins.

PROOF. Classify job i as C exactly when some retained preparation has its current identity, otherwise U . Use Equation (3) with $b = B - \#writs$. Immediately before a comparison acquires its monitor, the environment can preserve a current preparation or invalidate it with one fresh own-key write. Theorem 1's validation/cached inequalities apply. No monitor survives the boundary, so this works with colliding bins.

Fresh preparation performs the U -to- C credit transfer; computing from an old identity adds no current capability. Stale validation rejects without a write and leaves the semantic state unchanged.

A stale cached callback recomputes inside: its excess is $k_i + s_i - c_i = P_i$ in state U, or $k_i + s_i - m_i = w_i + P_i$ if another current preparation exists. Both dominate the protected option. Fresh callbacks use that option directly. Discarding the only current preparation increases potential; copies, stale preparations and preserving inspections cannot decrease it. Captured parents survive live-parent writes. Thus every operation admits a zero/one-write continuation preserving cost plus potential; telescoping gives the lower bound with at most B writes. The interface forbids using a shared bin to perform another job's work inside the same call.

For the upper bound, the serial controller freshly reads and prepares after every cheap failure. Its read-to-comparison intervals, including cached completions, are disjoint; each mismatch contains a distinct write. There are at most B mismatches. Counting them may overestimate remaining writes, conservatively, so the three-mode guarantee applies in either layout. $\square$

Operational accounting. Native kernels fold captured predecessor seeds and an immutable array of length $8w_i - |\text{Pred}(i)| > 0$, executing exactly $8w_i$ selected operations. The metric adds $8p_i$ per protected kernel, $8v_i$ per conditional call and $8k_i$ per callback, including mismatches. It scales the abstract objective by eight. Allocation, monitor waiting, initialization, background work and other API instructions are excluded; charges are not inferred from timing.

4.1 A Protected-Work Frontier Without Fitted Prices

Let W count all foreground kernel work, L the part performed inside atomic callbacks, and Q all foreground completion/validation calls, including failed conditionals. Reads, initialization and external-writer calls are excluded from Q . Here w_i is each complete kernel's mandatory work on every input, inside or outside protection, not merely an upper bound. Let Ω_k sum the largest k works, with $\Omega_0 = 0$ and $\Omega_n = \Omega$.

THEOREM 3 (PROTECTED WORK UNDER A CALL CAP). *Under Theorem 2's interface and environment, fix integers $B, r \geq 0$. Among programs that complete all jobs with $Q \leq n + r$ on every execution, the minimum worst protected work is*

$$L_3^*(B, r) = \Omega_{\min((B-r)_+, n)}, \quad L_2^*(B, r) = \begin{cases} 0, & B \leq r, \\ \Omega, & B > r, \end{cases}$$

with and without cached completion, respectively. Any DAG is permitted.

PROOF. At most r calls may fail to complete. Fresh attempts have disjoint intervals between read and comparison; each failure needs a distinct write. If $B \leq r$ they give $Q \leq n + B$, $L = 0$. Otherwise switch after r failures to cached completion: at most $B - r$ writes remain, giving the stated bound. All-protected gives the two-mode upper bound.

For the two-mode lower bound when $B > r$, reject each cheap call with a fresh own-key write, stopping writes after the $(r + 1)$ st rejection. This uses at most $r + 1 \leq B$ writes. An admissible program cannot incur that many noncompleting calls, so all its jobs complete under protection.

For three modes with $B > r$, target the $k = \min(B - r, n)$ largest jobs. Reject their cheap calls and force their cached calls to recompute by fresh own-key writes immediately before comparison; fresh callbacks already incur w_i inside. Stop writes after $r + 1$ target cheap rejections. Before a next write, at most r such rejections and $k - 1$ cached target completions have occurred, so that write still uses at most $r + k \leq B$ writes. Admissibility excludes $r + 1$ noncompleting calls. Thus every target completes with protected work, giving $L \geq \Omega_k$. Fresh identities invalidate retained preparations, and no inspection or completed callback retains a guard across the chosen boundary. $\square$

THEOREM 4 (ONE POLICY FOR EVERY WRITER BUDGET). *Let $\mathcal{U}_j(r)$ contain the j -mode programs that receive no B and complete with $Q \leq n + r$ in every finite-write environment. Set $U_j(B, r) = \inf_{P \in \mathcal{U}_j(r)} \sup_{\# \text{writes} \leq B} L(P, E)$. Then*

$$U_3(B, r) = L_3^*(B, r), \quad U_2(B, r) = \Omega \mathbf{1}[B \geq r].$$

One policy per mode family attains these values simultaneously for all B and positive works on any DAG, using only readiness and a global cheap-failure counter.

PROOF. Switch after r cheap failures to a fresh read/preparation with cached completion (three) or to fresh protected completion (two). This uses at most r extra calls universally. At $B < r$ no protection is needed; at $B = r$ cached completions after switching still match. Theorem 3 supplies the remaining upper and three-mode lower bounds.

For two modes when $B \geq r$, reject every cheap call with a fresh own-key write until r rejections, then stop. Only protected calls complete before this. Afterwards, another rejectable call could fail in a different finite environment, violating the call cap. Thus all jobs complete under protection with at most $r \leq B$ writes. That additional hypothetical write is absent from the lower-bound execution. Retained preparations and inspections cannot prevent a fresh write before comparison. $\square$

For equal works and $B > r$, the ratio is $n/\min(B - r, n)$, reaching n at $r = B - 1$. Linear prices $W + \lambda L + \kappa Q$ correspond to $p_i = \lambda w_i$, $v_i = k_i = \kappa$; these counts do not estimate throughput.

Work cost of simultaneous protection optimality. Among $\mathcal{U}_3(0)$ programs attaining the optimal worst- L curve for every budget, minimum worst W at B is $\Omega + \Omega_{\min(B, n)}$. All-cached attains it with $Q = n$, $W = \Omega + L$. To prove necessity, target the $k = \min(B, n)$ largest jobs. Each counted call must complete. Immediately before each target's first comparison/acquisition, insert a fresh write exactly if its kernel was previously executed. Every target then executes inside. If this uses $h \leq k$ writes, the simultaneous guarantee gives $\Omega_k \leq L \leq \Omega_h$. Positivity forces $h = k$: each target pays a prior and a protected kernel. Unknown B alone does not force this cost; the artifact gives a counterexample.

For general r , fix Theorem 4's three-mode policy, retrying each current job until completion. Minimum-work ready selection minimizes its worst W over fixed topological orders simultaneously for all B, r on any DAG. A heap constructs the order in $O(|E| + n \log(n + 1))$ operations using only weights and the DAG. The artifact supplies the work formula and block-exchange proof. This compares fixed orders, not adaptive or arbitrary causal programs, nor the full (W, L, Q) Pareto frontier.

5 All-Budget Compilation

Direct evaluation requires up to $(B + 1)N$ states, $N \leq 2^n$. Our binary-input construction removes budget enumeration; arbitrary DAGs still retain the ideal dimension.

5.1 Reducing the Self-Referential Choice

LEMMA 5 (SUBSET MONOTONICITY AND CHEAPEST VALIDATION). *If $T \subseteq S$, then $F(T, b) \leq F(S, b)$. For any minimum- c_i job $i \in \mathcal{A}(S)$, its cheap-validation option is no larger than that of any other available job.*

PROOF. For monotonicity, simulate a policy for S , completing omitted jobs without invalidation and with nonnegative excess. Relay retained jobs' actions and outcomes. Every virtually available retained job is physically available; omitted jobs spend no token. Physical cost never exceeds virtual cost. Equivalently, induction on $|S|$ and b compares corresponding terms of Equation (2).

At $b = 0$ all cheap-validation values are zero. For $b > 0$, let $j \neq i$ be available. It remains available after removing i . Choosing cheap validation of j in $S - i$ and using monotonicity gives

$$F(S - i, b) \leq \max\{F(S - \{i, j\}, b), c_j + F(S - i, b - 1)\} \leq \max\{F(S - j, b), c_j + F(S, b - 1)\}.$$

The other branch $c_i + F(S, b - 1)$ is also bounded by the last maximum. Their maximum is the cheap-validation value for i . $\square$

For a child curve f with $f(0) = 0$, define the insertion operator

$$(I_s f)(0) = 0, \quad (I_s f)(b) = \max\{f(b), s + f(b - 1)\}, \quad b \geq 1.$$

Fix a cheapest available i , put $g(b) = F(S - i, b)$, $c = c_i$, and

$$h(b) = \min_{j \in \mathcal{A}(S)} \{P_j + F(S - j, b), \delta_j + I_{s_j} F(S - j, \cdot)(b)\}. \quad (4)$$

At zero, $h(0) = \min_j \{P_j, \delta_j\}$ is allowed to be positive. Monotonicity and feasible completing modes give $g(b) \leq F(S, b) \leq h(b)$. Equation (2) becomes

$$f_0 = 0, \quad f_b = \min\{h(b), \max\{g(b), c + f_{b-1}\}\}, \quad b > 0. \quad (5)$$

This is a specialization of a reflected recurrence [27], not a new general dynamic-programming principle. Its particular child curves permit the following closed form.

LEMMA 6 (CONCAVE FLOOR AND CAP). *Suppose g, h are nonnegative, nondecreasing and discrete concave, $g(0) = 0$, and $h \geq g$. Define*

$$D(0) = 0, \quad D(b) = \sum_{t=1}^b \max\{c, \Delta g(t)\}, \quad c > 0.$$

Then Equation (5) has solution $f_b = \min\{h(b), D(b)\}$, which is nondecreasing and discrete concave.

PROOF. Since $\Delta g(t) - c$ is nonincreasing,

$$D(b) = \max_{0 \leq t \leq b} \{g(t) + c(b - t)\} = \max\{g(b), c + D(b - 1)\}.$$

Induct on b . If $h(b - 1) \geq D(b - 1)$, substitute this identity directly. Otherwise choose a maximizer t for $D(b - 1)$. Because $h(b - 1) \geq g(b - 1)$, the strict inequality requires $t < b - 1$. From $h(t) \geq g(t)$,

$$h(b - 1) - h(t) < c(b - 1 - t).$$

Concavity makes the next difference of h smaller than c . Thus the recurrence selects $h(b) < c + h(b - 1)$. The same block inequality for g makes $D(b) = D(b - 1) + c > h(b)$. This proves the formula in both cases, including the origin. Flooring preserves nonincreasing nonnegative differences. Pointwise minima preserve nondecreasingness and discrete concavity: minimize $2u_b \geq u_{b-1} + u_{b+1} \geq \min_u u_{b-1} + \min_u u_{b+1}$ over the candidate curves. $\square$

5.2 Integer Insertion and Supporting Slopes

We prove the cached maximum's concavity through its marginal structure.

LEMMA 7 (INTEGER MARGINAL INSERTION). *Let a bounded curve with $f(0) = 0$ have a nonnegative supporting representation $f(b) = \min_{d \in D} (db + q_d)$ for all integer $b \geq 0$, with finite $D \subseteq \mathbb{R}_{\geq 0}$ and $0 \in D$. For $s > 0$, define*

$$Q_s = \max_{b \in \mathbb{N}} \{f(b) - sb\}.$$

Then

$$I_s f(b) = \min \left\{ sb + Q_s, \min_{d \in D} (db + q_d + (s - d)_+) \right\}. \quad (6)$$

Its marginal sequence is that of f with one additional marginal s inserted in decreasing order.

PROOF. The marginals of f are nonincreasing and eventually zero. Let t be the number at least s . For $b \leq t$, $I_s f(b) = f(b)$; for $b > t$, it equals $s + f(b - 1)$. This proves insertion and gives $Q_s = f(t) - st$.

Every line on the right of Equation (6) majorizes this inserted curve. Before t , unchanged slopes at least s already majorize f ; smaller slopes receive a nonnegative shift. After t , shifting a smaller slope by $s - d$ exactly forms s plus its old value at $b - 1$, while a larger slope provides at least the same increase. The contact line $sb + Q_s$ majorizes both portions.

For attainment, an active supporting slope d_b at integer b satisfies

$$\Delta f(b + 1) \leq d_b \leq \Delta f(b),$$

where only the first inequality is needed at $b = 0$. At $b < t$ an active line therefore has slope at least s and is unchanged. At $b = t, t + 1$ the contact line attains. At $b \geq t + 2$, an active line at $b - 1$ has slope smaller than s and its shifted version attains. The cases $t = 0$ and $f \equiv 0$ follow from the same contact line. $\square$

The contact is over integers. For $f(b) = \min\{100, b + 1, 3b\}$ and $s = 2$, $I_s f(1) = 2$. Replacing the integer contact 0 by the continuous supporting envelope's supremum $1/2$ gives 2.5 in Equation (6). Computing a real contact and rounding afterwards is not the stated identity.

The floor operation also preserves a finite input-slope basis. If $f(b) = \min_d(db + q_d)$ and $c > 0$, then

$$\sum_{t=1}^b \max\{c, \Delta f(t)\} = \min \left\{ cb + Q_c, \min_{d>c} (db + q_d) \right\}. \quad (7)$$

Let $a = \#\{t \geq 1 : \Delta f(t) > c\}$, the first integer maximizing $f(b) - cb$. The floor equals f through a , and equals $cb + Q_c$ thereafter. At $b < a$ every attaining line has slope above c ; at and after a , each retained larger-slope line stays above the contact line. This definition also handles a plateau with marginal exactly c , or $a = 0$, without special rounding.

THEOREM 8 (ALL-BUDGET BASIS AND ENCODED SIZE). *For every residual set S , $F(S, \cdot)$ is nonnegative, nondecreasing and discrete concave, with eventual value $\sum_{i \in S} P_i$. It has a supporting representation*

$$F(S, b) = \min_{d \in \mathcal{D}_S} (db + q_{S,d}), \quad \mathcal{D}_S \subseteq \{0\} \cup \{c_i, s_i : i \in S\}. \quad (8)$$

There are at most $2|S| + 1$ lines and at most $4|S|$ positive marginal runs. All reachable-state curves and action data can be constructed and checked in $2^n \text{poly}(L)$ bit time, with no iteration through the magnitude of B .

PROOF. Induct on $|S|$, starting with the zero line for the empty set. By Lemma 7, each cached term in h shifts existing intercepts and adds only slope s_j . Protected terms shift intercepts by P_j . Their minimum is a concave supporting envelope. Lemma 5 gives $g \leq F \leq h$ independently of the desired formula, so Lemma 6 applies. Equation (7) adds only the cheapest available c_i . Union the line families and retain the smallest intercept per slope. This proves Equation (8), concavity and nondecreasingness.

Protecting all jobs gives the bound $\sum_i P_i$. A bounded finite lower envelope of nonnegative slopes is eventually constant; let its limit be L_S . By smaller-set induction, sufficiently large budgets in Equation (2) satisfy

$$L_S = \min_{i \in \mathcal{A}(S)} \left\{ P_i + \sum_{j \in S-i} P_j, \quad c_i + L_S, \quad \delta_i + s_i + \sum_{j \in S-i} P_j \right\}.$$

Here $c_i > 0$ and $\delta_i + s_i = P_i + w_i > P_i$, forcing $L_S = \sum_{i \in S} P_i$.

At most $2|S|$ distinct positive slopes give at most that many continuous-envelope changes. Integer sampling adds at most one bridging unit interval per noninteger change, so at most $4|S|$ positive runs suffice. This is a conservative upper bound, not a tightness claim. Summing $4|S| + 1$ over all subsets gives $(2n + 1)2^n$ profile entries, with at most $n2^n$ action identities.

Generated intercepts obey $0 \leq q_{S,d} \leq \sum_{i \in S} (P_i + w_i)$. A protected shift adds P_i ; a cached shift adds at most $\delta_i + s_i = P_i + w_i$; a new contact is bounded by child saturation; flooring retains old intercepts. Contacts require no unbounded search: for profile triples (a, y, d) ,

$$Q_S = \max_{(a, y, d) \in \text{profile}(f)} (y - sa).$$

Each piece is affine and the final constant tail decreases after subtracting sb , so a listed breakpoint attains the maximum. Intercepts, integer slopes and crossing-neighbor positions consequently have polynomial bit length in L . Each state uses polynomially many shifts, contacts and envelope intersections on $O(n)$ lines. Affine-partition checking below has polynomially many such endpoints per state. Enumerating at most 2^n states proves the bit bound. $\square$

5.3 Emitted Policies and Independent Checking

Profiles store affine pieces (a, y, d) with a constant tail. Both constructors process reachable upsets child-first, using floor/cap or supporting-basis operations without enumerating budgets. Runtime selection evaluates Equation (2) with $O(n)$ indexed profiles, chooses a deterministic argmin and decrements allowance on mismatch. Query cost is polynomial in n , stored size and $\log(B + 1)$; optimal ties need not imply pointwise dominance.

The checker imports no constructor. It reconstructs readiness and the reachable domain, validates raw and derived prices, baseline, available-job lists, cheapest-fast IDs, origins, continuity, decreasing slopes and tails, and verifies Bellman equality. Partition at own/child starts and their one-step shifts. Within each interval all atomic expressions are affine. Checking endpoints and floor/ceiling neighbors of all crossings proves equality throughout; check the constant final interval directly. Compare the supporting basis with its emitted curve by the same affine method.

Induction on $|S| + b$ certifies values and the mathematical argmin. Executable selection, arithmetic, parsing and the operational abstraction remain trusted or separately tested. Earlier routing-binding failures remain recorded in the artifact and repaired by RQ1's field checks. Polynomial-per-state bounds concern generated certificates; arbitrary encodings additionally cost their supplied length. Checking does not establish application-price adequacy.

What a requested-value query can skip. Large b need not require a profile. Classical concave allocation [12] gives the following charged-model bounds:

$$\max_{\substack{b_i \in \mathbb{N} \\ \sum_i b_i \leq b}} \sum_{i \in S} \min(P_i, b_i c_i) \leq F(S, b) \leq \min_{t \in \{0\} \cup \{c_i : i \in S\}} \left(\sum_{c_i > t} P_i + bt \right). \quad (9)$$

For the lower bound, assign private failure budgets b_i . Protected or cached-failure completion pays at least P_i ; any other completion after exhausting its private allowance has already paid $b_i c_i$ in cheap failures. Their total allowance is at most b . For the upper bound, protect jobs with $c_i > t$ and retry the others in any topological order. Their failure costs are at most t . The allocation merges compressed marginal runs: $\lfloor P_i / c_i \rfloor$ copies of c_i , then $P_i \bmod c_i$ if nonzero. It never iterates to b . Equality certifies a value directly, and $b \geq \sum_i \lceil P_i / c_i \rceil$ guarantees the constant tail $\sum_i P_i$. Our requested-value comparator uses these tests at every subproblem and separately certifies equality

leaves and concave peaks. It returns requested values, whereas the full artifact additionally supplies all-budget profiles and routing data.

6 Compact Compilation, a Sharp Ratio and the General Barrier

The general compiler handles arbitrary dependencies and prices. A price condition, however, permits a stronger result even with positive call charges and an essential cached mode.

6.1 Reducing Mixed Charges to Two Fees

For a concave suffix f with $f(0) = 0$, write

$$R_{a,q}f(b) = f(b) + \min \left\{ q, \sum_{\ell=1}^b (a - \Delta f(\ell))_+ \right\}. \quad (10)$$

This is the two-fee operator with failure price $a > 0$ and protected premium $q \geq 0$: by Lemma 6, it equals $\min(f + q, L_a f)$, where $L_a f(b) = \max_{0 \leq j \leq b} \{f(j) + a(b - j)\}$. It is monotone in f .

LEMMA 9 (TWO-FEE ORDERING AND PACKING). *Independent two-fee jobs admit a fixed optimal nondecreasing- a completion order, including against adaptive choices. Composing their R operators in this order gives their value H . Reverse composition fills marginal deficits below the next a : all positive suffix marginals except possibly the last are at least a . Each job adds at most two runs. Every sorted suffix is the root curve capped at its remaining premium sum.*

PROOF. Let G be the optimum restricted to the sorted order. Inserting a protected job i at its sorted position gives $G(S, b) \leq q_i + G(S - i, b)$ without spending a token. Induct on $|S| + b$. The first sorted job has minimum failure price; the proof of Lemma 5, with only protected and cheap modes, makes its cheap option no larger than any other's. The smaller-set and previous-budget values equal G by induction. Thus G is bounded above by every unrestricted Bellman option, while feasibility gives the reverse inequality.

Equation (10) preserves marginals at least a and raises smaller ones toward a using at most q mass. In reverse sorted order, only the final positive singleton can be below a . Raise it first; write any remaining mass as $ha + r$, $0 \leq r < a$, and append h copies of a and one r if nonzero. Constantly many run operations suffice. Removing the added tail mass recovers the previous suffix, proving the cap identity. $\square$

THEOREM 10 (EXACT MIXED-PRICE REDUCTION). *Suppose $p_i \delta_i = 0$ for every job, with the original prices from Equation (1), and set*

$$t_i = \begin{cases} \min(c_i, s_i), & \delta_i = 0, \\ c_i, & \delta_i > 0 \quad (\text{hence } p_i = 0). \end{cases} \quad (11)$$

For independent jobs, the charged excess $F(S, b)$ equals the two-fee optimum $H(S, b)$ with prices (t_i, P_i) at every integer budget. A fixed nondecreasing- t completion order is optimal. The same equality holds for a DAG satisfying $t_u \leq t_v$ on every edge $u \rightarrow v$, with ties ordered topologically. The total charge remains $\sum_{i \in S} c_i + H(S, b)$.

PROOF. Marginal insertion (Lemma 7) commutes with deficit filling when $s \geq a$:

$$R_{a,q} I_s f = I_s R_{a,q} f. \quad (12)$$

Indeed, insertion of s precedes every marginal below a and adds no deficit; $R_{a,q}$ changes only those later marginals, raising them at most to a .

If f is a sorted suffix with effective costs at least t , $s \geq t$ and $q < s$, then $R_{t,q}f \leq I_s f$. Before the final partial marginal the deficit is zero. At that marginal the increment is at most $t - \Delta f(b) \leq s - \Delta f(b)$; after saturation it is at most $q < s$. These cases also show $R_{s,q}f = \min(f + q, I_s f)$ when $t = s$.

For $\delta_i > 0$, $p_i = 0$, so cached completion is dominated: $\delta_i + I_{s_i}f \geq P_i + f$. The ordered charged operator is exactly R_{c_i, P_i} . For $\delta_i = 0$, $P_i = p_i < s_i$, and it is $\min(f + P_i, L_{c_i}f, I_{s_i}f)$. The preceding suffix inequalities make this R_{t_i, P_i} , both when $t_i = c_i$ and when $t_i = s_i$. Thus the sorted charged policy attains H .

For a matching lower bound, the two-fee Bellman equation bounds $H(S, b)$ by $P_i + H(S - i, b)$ and by $\max\{H(S - i, b), c_i + H(S, b - 1)\}$, since $t_i \leq c_i$. These cover protected and cheap choices. Positive- δ_i cached choices are covered by protection dominance. For a zero- δ_i job at any sorted position, write $H(S) = AR_{t_i, P_i}f$ and $H(S - i) = Af$. Every operator in the preceding composition A has cost at most $t_i \leq s_i$. Monotonicity, the suffix inequality and Equation (12) give

$$H(S) \leq AI_{s_i}f = I_{s_i}Af = I_{s_i}H(S - i).$$

Hence every original charged Bellman option is at least H ; induction on $|S| + b$ proves the lower bound against adaptive choices. Finally, removing DAG edges gives an independent lower bound, and a compatible sorted topological policy attains it legally. $\square$

The class mixes $k_i \leq v_i$ jobs with zero-premium jobs. With $v_i = k_i = \kappa$ and $p_i = \lambda w_i$, $t_i = \min(w_i + \kappa, (1 + \lambda)w_i)$ increases with w_i , so work order suffices. Effective prices preserve cached choices; the total-cost baseline remains $\sum c_i$.

6.2 A Checked Cursor Policy

Minimum- t Kahn traversal finds a sorted topological order when the edge condition holds. Apply Lemma 9's packing to (t_i, P_i) , retaining the original charged input. For sorted suffix S_k , root curve H and $Z_k = \sum_{i \in S_k} P_i$, store

$$F(S_k, b) = \min\{H(b), Z_k\}. \quad (13)$$

The artifact stores $O(n)$ entries beyond the graph for its $n + 1$ policy suffixes; the general artifact covers all reachable unfinished sets.

The checker independently recomputes reduction, baseline and order; validates origin, positive integer run lengths, decreasing heights, zero tail and total mass $H(\infty) = Z_0$; and checks the two-fee Bellman equations. For adjacent caps $z = Z_k$, $a = Z_{k+1}$, let u and v be the first budgets where H reaches a and z . Check the uncapped prefix, the boundary u , the constant marginal t_i between u and v , and saturation at v ; concavity extends marginal endpoint checks across each interval. Zero premiums require identical adjacent caps. The terminal cap is zero. These conditions verify all budgets by suffix induction; Theorem 10 supplies the charged/global-optimality reduction. Additional original-mode boundary checks are consistency controls, not a substitute for that proof.

Construction takes $O(|E| + n \log(n + 1))$ arithmetic operations. Checking a supplied root with r runs takes $O(|E| + n \log(r + 2) + r)$, plus its encoding length. Generated roots have $r \leq 2n$, giving the same asymptotic bound. Binary operand lengths still count. Prefix lengths/areas give $O(\log(n + 1))$ value queries. At a cursor, evaluate the three original branches in Equation (2) for its fixed next job using adjacent caps. Cheap failure retains the cursor; cached failure spends one token and advances it. All paid charges remain original. Inputs failing the price or edge condition use the general charged compiler.

The conditions are sufficient. For independent $w = (1, 1, 1, 2)$, $p_i = w_i$, common $v = 5$, $k = 6$, and $B = 1$, ascending work order costs 31, while order $(2, 1, 1, 1)$ and the adaptive optimum cost 30; here $p_i \delta_i > 0$. Even with $\delta_i = 0$, sorting by c_i can fail: $(w, p, k, v) = (5, 6, 5, 8)$, $(5, 4, 6, 8)$ gives $c = (10, 11)$ but $t = (10, 9)$, and $B = 1$ totals 31 versus 30. Both refutations remain in the artifact.

6.3 A Sharp Bound on Independent-Job Savings

The third mode's scalar benefit is bounded even outside the compact class. Write F_2, F_3 for the two- and three-mode excess values and $A = \sum_i c_i$.

THEOREM 11 (SHARP INDEPENDENT-JOB RATIO). *For independent jobs with arbitrary charged prices and any integer $B \geq 0$,*

$$\frac{A + F_2(B)}{A + F_3(B)} \leq \frac{1 + \sqrt{2}}{2}.$$

The constant is a supremum, already approached with common $v_i = k_i$ and $p_i = \lambda w_i$.

PROOF. Put $X = F_2(B)$ and $Y = F_3(B)$. Equation (9)'s private-budget argument gives $D(B) = \max_{\sum b_i \leq B} \sum_i \min(P_i, b_i c_i) \leq Y$, including when $\delta_i > 0$.

Pack the two-mode premiums in reverse sorted- c order (Lemma 9), labelling each contribution by its job. A final unit column containing job i 's mass has height $h \leq c_i$, since later caps are no larger. If x_i is that job's mass in the first B columns, then $\sum x_i = X$, $0 \leq x_i \leq P_i$, and $\sum x_i/c_i \leq B$: each nonzero column contributes at most its mass divided by its height, namely one.

Let ϕ_i linearly interpolate $\min(P_i, bc_i)$ at integer b . Under $z_i \geq 0$ and $\sum z_i \leq B$, the maximum of $\sum \phi_i(z_i)$ is $D(B)$: select the B largest nonincreasing unit slopes, preserving tied prefixes. For $z_i = x_i/c_i = \ell_i + \rho_i$, $\ell_i \in \mathbb{N}$, $0 \leq \rho_i < 1$, $z_i \leq P_i/c_i$ implies $\phi_i(z_i) \geq c_i(\ell_i + \rho_i^2)$. With $\alpha = \sqrt{2} - 1$,

$$\ell_i + \rho_i^2 - 2\alpha(\ell_i + \rho_i) + \alpha^2 = \ell_i(1 - 2\alpha) + (\rho_i - \alpha)^2 \geq 0.$$

Thus $Y \geq 2\alpha X - \alpha^2 A$; $1 - \alpha^2 = 2\alpha$ gives the ratio. For sharpness take $n = M^2$ equal jobs with $w = 1$, $v = k = M$, $p = \lfloor \alpha M \rfloor$ and $B = \lceil np/(M+1) \rceil$. Packing gives $F_2 = \min(np, B(M+1))$ and $F_3 = \min(np, B(p+1))$; their total-cost ratio converges to the bound. $\square$

Hence the supremum relative total-cost saving is $3 - 2\sqrt{2} \approx 17.16\%$ on independent inputs. The argument uses sorted independent-job packing; it asserts no such bound for arbitrary DAGs.

6.4 Hardness Outside the Compatible Class

Set $v_i = k_i = 0$. Then $c_i = w_i$, $\delta_i = 0$, $P_i = p_i$, and the cached mode is dominated by a cheap attempt followed by immediate protection on failure. Equation (2) reduces to

$$V(S, b) = \min_i \{p_i + V(S - i, b), \max[V(S - i, b), c_i + V(S, b - 1)]\}.$$

At $B = 1$, a topological order π and the protected jobs H on its no-failure path have worst excess

$$C(\pi, H) = \max \left\{ \sum_{j \in H} p_j, \max_{i \notin H} \left(c_i + \sum_{j \in H: j \prec_\pi i} p_j \right) \right\}. \quad (14)$$

After the one possible failure, every remaining job finishes without excess. Minimizing this expression is exact, so an initially fixed order suffices at one failure.

THEOREM 12 (SINGLE-FAILURE HARDNESS). *The zero-fee subcase of deciding $F(J, 1) \leq K$ is NP-complete with at most two DAG layers, maximum indegree two, unit premiums and at most two distinct positive failure prices.*

PROOF. Equation (14) verifies order/mode bits in polynomial time. For nontrivial CLIQUE [33], $k \geq 2$, $m \geq \ell = \binom{k}{2}$, add an isolated vertex so $n > k$. Set $K = n + m - \ell > k$; give vertices $(c, p) = (K + 1, 1)$ and endpoint-dependent edges $(K - k, 1)$. A clique gives value K : protect its vertices, run its ℓ edges fast, protect the remainder. Conversely, value $\leq K$ forces all vertices protected and every fast edge among the first k protected vertices. No-failure cost $n + m - f \leq K$ requires at least ℓ fast edges, hence a clique. All numbers are $\leq n + m + 1$. $\square$

This does not establish NP membership for binary B or require ideal enumeration. A w -chain cover bounds ideals by $\prod_\ell (n_\ell + 1)$, polynomial for fixed w (XP).

Dependencies can require adaptive ordering even without fees: $c = (3, 7, 5, 1)$, $p = (2, 2, 6, 2)$ and edges $0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 3$ give 24 versus 26 at $B = 2$. Every fixed order gets optimal adaptive modes and knows B . This constructed separation does not establish typical benefit.

7 A Source-Derived Hierarchical Snapshot Case

Deephaven retries optimistic hierarchical snapshots and can fall back to shared-lock execution. The inspected server API extracts key directives (K), then materializes an ordered viewport and expanded size (V). The Web client's static keys do not retry; our two-refreshing-stage configuration uses the supported server API [4].

A source-derived resource bound. At source commit 6367313a, fix a 31-row balanced hierarchy, immutable ID/Parent columns, directive keys 0, 1, 2, a 16-row single-range viewport and four output columns. Selected-operation bounds are $W_K = 4$, $W_V = 104$: four directive links, four parent lookups, no unlinks, 32 node operations and 64 output-cell operations. Earlier inputs preserve Actions; a later study alternates node 1 between Contract and ExpandAll, keeping the others expanded, and changes payloads.

The bounds include mid-body updates. Fixed keys prevent insertion/rehashing; each compact-range iterator retains its bounds when previous-value references advance. Actions choose a full or contracted subtree, without adding ancestors or removing linked children; payloads preserve traversal lengths. Expansion uses 32 node operations including null leaves; contraction uses 17, totaling 89 selected operations. Inconsistency exits without restart. These pre-outcome source conditions give cap 104, not a trace maximum.

Use unit unprotected weight and protected weight $1 + \lambda$, supplying zero call charges and $p_i = \lambda W_i$. Short/aborted bodies charge executed prefixes. Setup, allocation, waiting, API instructions and other operations are excluded; this is selected work, not CPU time. Periodic real-time-tree updates supply no finite bound [5]. Our caller instead owns at most q public EventDrivenUpdateGraph.requestRefresh invocations [4, 6], each running a complete native cycle. No free foreground operation certifies that future invocations have ended.

The evaluated update contract. A source latch exposes the interval from START to mutation. Releasing it finishes that refresh through notifications and COMPLETE. Decisions observe synchronized transitions: serial inconsistent bodies charge distinct START or completion boundaries, giving at most $2q$ failures. Shared-lock execution drains an open cycle first. Partially notified interleavings are excluded; native notification state is richer than raw clock parity.

An imported profile for $K \rightarrow V$, $B = 2q$, bounds bodies by the zero-fee value plus $W_K + W_V$. Inconsistency charges its failed-body cap and consumes allowance; success charges actual selected work. The materialized K directives persist for V even if live Action changes. This gives an upper guarantee; the native graph cannot realize every abstract adversarial choice.

At $\lambda = 3$, $q = 2$, the cap is 424. One fast-tie script attains 424 via four full failures over two split cycles. A local rule with the same allowance costs 432: it protects when local failure exposure reaches the premium. Attainability by that policy is not whole-system minimax optimality. A phase-aware mode game drains an open cycle by protection and has value 420, exposing slack in the $2q$ abstraction.

Integration boundary. Source hooks check all normal returns and reject foreign/skipped calls, exceptions and bound violations. The upstream two-attempt replica uses the same harness with elapsed-time fallback disabled. Refreshes use the public API, but the caller must own all of them and

supply q : periodic rate, thread count and hidden writers do not provide it. Automatic applicability and calibration remain open.

8 Implementation and Evaluation

We test game/policy agreement (RQ1), the source case (RQ2), and construction costs (RQ3). Studies are authored and exploratory: reused cases are identified and new generation rules precede outcomes. Every input, failure, timeout, invalid unit and correction remains recorded; none is a held-out application population.

Experiments use an Apple M2/24 GiB, Python 3.10.9 (later 3.12.14) and Homebrew OpenJDK 17.0.19. Retained ConcurrentHashMap.java matches the cited source; the binary was not rebuilt. Hashes bind inputs, code, profiles, commands and traces. Separate Python checkers validate outputs/equations. Codex and other assistants supported models, code, source analysis, proofs and writing. These author-side checks are neither mechanical proof nor independent blind review; no human or simulated-human study was performed.

8.1 RQ1: Charged Values, Routing and Native Calls

The revised general compiler/checker agrees with direct values on 3,183 observed one-to-eight-job inputs, including 128-bit prices, after serialization/reloading. All finish within five seconds/unit and 300 seconds/campaign; 11 certificate controls pass. The compact compiler agrees on 19,485 inputs (15,727 observed, 3,758 new), with 16,770 compact routes, 2,715 fallbacks and 527,409 checked policy branches at budgets 0..4. Its 29 certificate/runtime and 14 CLI controls retain the repaired Python numeric-alias and null-input binding defects.

On 34 observed native inputs, 2,278 policy-path replays and 816 concurrent runs pass price, policy, output and parent checks; all 340 replay-cell maxima equal the compiled value. Concurrent runs contain 74 failures and 3,060 writes. Four native controls reject substituted parents and omitted charges. Earlier checkers accepted false failure epochs, substituted outputs and an impossible writer-log order. The replacement searches foreground/writer microprograms with reads, allocations, comparisons, publications, bin locks and submit/join; another checker replays a witness sequence. All original 3,098 records and 12 record/11 sequence controls have expected outcomes. The check establishes only existential model feasibility under shared source translation.

The compact adapter changes only loading/selection. Six of 36 screened fixtures qualify; all 30 inapplicable rows remain. With 20 new mixed-price fixtures, its 502 path replays and 104 concurrent runs pass, and all 208 group maxima match. Four native, eight record, five sequence and nine parser controls pass. A positive- δ fixture retains cap 8 versus two-mode 9. Native jobs/prices are bounded (at most five jobs); scalable arithmetic uses Python integers. All ordinary units above succeed without timeout, invalid or unstarted cases.

Separate resource guarantees. Eight shapes, two common call weights and six selectors produce 2,056 path replays, 384 concurrent runs and two native controls, all expected. All 576 group maxima of W, L, Q and per-execution $R = W + L + 8\kappa Q$ match direct equations and independent source-counter reconstruction. Scalar three-mode beats two-mode in only two of 96 cells, both layouts of Table 1; 94 tie. Scalar optimization need not minimize L : a decreasing five-job chain gives $L = 48$ for the $Q = n$ scalar selector versus cached-all's 40, at equal $Q = 5$ and worst cost 200.

A missing price precondition let a checker accept an out-of-class false constant tail. A strict recheck validates 96 tables and 2,442 saved decisions; five controls reject unrelated errors presented as the intended native rejection. The recorded live-parent rejection was correct.

Direct minimax L equations agree for all 265,629 budget/slack pairs from 5,421 weighted DAG inputs. Budget-aware threshold policies produce 8,800 path replays, 512 concurrent runs and two

native controls; all 1,024 group maxima attain the bound. A separate budget-unaware selector receives only a cloned DAG, mode family and r . It produces 17,200 path replays, 512 concurrent runs and two controls; all 1,280 groups attain Theorem 4, including its $B = r$ separation. Its bytecode reads no writer, price or table state. Each of these three resource studies passes eight record, five sequence and four parser controls. Native budgets end at 2, 3 and 4; no table reaches the unaware selector. Post-outcome analysis of all 17,200 path records/1,280 groups also matches its exact component-wise work formulas, including 160 groups at $r = 0$; these add no measurements. A separate order check reuses the 5,421 inputs and adds 96 authored cases, an empty case and one known illustration. All 86,662 legal orders/2,773,184 budget-slack pairs confirm the greedy minimum and direct work equations; seven artifact controls pass.

Ratio checks cover 13,295 common-price inputs/172,835 roots and seven growing tightness families. Another 622 general-price inputs at three uniform scales give 1,866 checks/24,378 roots and 3,732 large-budget queries, all successful. Scaling is not independent sampling. The 65,536-job family reaches ratio 1.20545; the proof gives the limit.

A failed mean-time calibration. JMH 1.37 [2] measures nine lookup/preparation/call-outcome wrappers, four payload lengths and two layouts: 72 cells, 144 forks and 720 measurements, all complete without retry or exclusion. Each fork uses three 200-ms warmups and five 200-ms measurements. A fixed nonnegative fit maps preparation to w , conditionals to v , validation-only to k , and fresh protection to $w + k + p$; cached outcomes are validation targets. Only two of eight settings meet the pre-fixed 20% maximum-relative-residual criterion. Every unconstrained residual is negative and every fitted p is zero. Of 1,152 fixed two-job/budget/weight roots, 72 fall in admitted settings and none gains from the third mode; 1,080 remain outside the fitted model. Different reuse, assignments and lifetimes prevent isolating protection overhead; short warmup and fork differences further limit inference. Mean-time benefit remains unestablished.

8.2 RQ2: Imported Bounds and an Existing Public Graph

The prefix/END study has three protection weights, 12 scripts and six policies: all 216 native/trace cells succeed. They contain 153 cycles and 223 inconsistent bodies, but zero prefix aborts: positive-depth ExpandAll bypasses the leaf-null check. Source review rejected a proposed alternate input whose intended abort mechanism was absent. The protected-tie compiler beats the stronger local rule in only three cells (424 versus 432 twice; 420 versus 432 once). It loses six of 36 cells to the two-attempt replica and four to the unknown-budget threshold; the fast-tie policy also loses a cell, 216 versus 116. Full matrices remain recorded.

With $\lambda = 3$, four scripts and three policies, all 12 public-graph native/trace cells succeed: 11 public refreshes yield 19 inconsistent bodies. The unchanged imported profile is 208 bytes. Checks bind row order, immutable structure, consistent payload epochs, body caps and refresh counts. Under the cross-stage script, compiler/local/replica charges are 424/432/632, with refresh counts 2/0/2; without updates all cost 108. Protection can suppress later script cuts, so realized writer workloads are policy-dependent.

Changing Actions extends this to all 12 scripts and three policies: all 36 cells succeed, with 33 public refreshes. Separate DFS reconstruction verifies 28 full-tree and eight contracted results. Two executions retain epoch-1 contracted K while returning epoch-2 payloads after live Action re-expansion; the successful-body ledger binds the captured epoch. The compiler beats/ties the same-budget local rule in 3/9 scripts and the replica in 6/6. One script costs 275/432/542, below compiler body/model caps 320/424.

A stronger checker verifies exact selected-work totals, including normally returned inconsistent bodies, previous-value choice during paused sources, immediate eligible cuts and synchronous

refresh blocking. All 36 records pass; 17 corruptions are rejected and four positive controls pass. The old checker missed eight of those corruptions. Ten upstream/MAX-key regressions also pass. An exploratory phase-aware study has 108 successful cells: protection can drain an open cycle, and already-notified sources can use current values while others retain previous values. Thus the 2q compilation remains an upper-bound application. Every study retains its full outcomes, source conditions and timing records; none establishes the unobserved abort path or whole-system native minimax optimality.

8.3 RQ3: Construction Cost and Stronger Value Queries

The general comparison has 24 authored inputs: four topologies (chain, four chains, fence, independent), $n = 8, 12$ and three price families: small heterogeneous, premiums scaled by 2^{32} , or all prices scaled by 2^{96} . Workloads request the first 1, 4 or 12 budgets of (1, 2, 4, 8, 16, 32, 64, 256, 4096, 65536, 2^{32} , 2^{64}). ALL constructs/writes/reloads/checks all profiles; PEAK memoizes concave peaks and checks requested-value certificates; DP uses two Bellman rows and stops at an all-state fixed point. Cold costs include imports through output/checking, capped at 4 seconds/1 GiB.

Initial successes are 63/72 ALL, 62/72 PEAK, 64/72 DP; eight generic AND-OR units also succeed (224 units/1,256 roots total). Post-outcome elementary bounds strengthen PEAK/DP in a separate 144-unit/816-root campaign to 64/72 each. All others time out; shared values agree. Conformance on 76 inputs/532 values/304 generic roots and 11 controls passes. On 14 common all-12 successes, PEAK/ALL median time changes from 7.16 to 0.576; revised DP/ALL is 0.457. These are separate single-sample campaigns. Seven long-premium ALL cases finish while both value methods time out; uniform scaling preserves budget geometry and has no such advantage. All nine ALL timeouts are independent 12-job cases. Thus elementary bounds remove most of PEAK's original disadvantage, while full enumeration remains costly.

Compilation on the compatible charged class. Campaign A fixes 42 new compatible inputs: $n = 8, 12, 32, 128, 512, 4096, 32768$, independent or height-two DAGs, and three numerical scales as above. All have $\delta = 0$ and edges respecting derived t order. One- and 12-budget workloads produce 84 units/method. PACK includes artifact writing/reloading/checking and runtime indexing; ROOT ablates checking/policy output using the same reduction, not a separate prior algorithm. PEAK/DP use adjacency validation and grouped bound scans.

All 420 cold units record within 1,315.17 seconds under 8-second/1-GiB caps (Table 3). PACK's 12 largest-input runs take 0.837–1.151 seconds; peak RSS is 122,044,416 bytes and maximum artifact 8,084,847 bytes including input. Checking/indexing dominates. ROOT's maximum is 0.241 seconds. On common successes, median PEAK/PACK and DP/PACK ratios are 0.860 and 0.792: requested values can be cheaper.

Table 3. Compact-class campaigns. Each cell is success/timeout/memory limit. A: 84 units per method; B: 36 units per method with $2n$ root runs. All outcomes are retained; returned services differ.

Method	A	B	Returned service
PACK	84/0/0	36/0/0	Checked all-budget cursor policy
ROOT	84/0/0	36/0/0	Same-reduction values, no checker
ALL	22/56/6	0/30/6	Checked all reachable unfinished sets
PEAK	34/50/0	18/6/12	Checked requested root values
DP	34/46/4	18/15/3	Requested values only

Campaign A's large curves have only 40 runs. Campaign B fixes 18 mixed-price inputs with $n = 512, 4096, 32768$, the same topologies/scales and exactly $2n$ root runs; long cases include 2^{32} -length plateaus. Final query budgets become $(2^{32} - 1, 2^{32}, 2^{32} + 1, 2^{64})$. All 180 units record under the same caps. PACK/ROOT complete all 36 each; PEAK/DP complete only 18 one-budget workloads. Every PACK profile equals the fixed run sequence and all successful queries match the fixed closed form. At 32,768 jobs, PACK takes 1.333–1.603 seconds for 65,536 runs, with peak RSS 152,911,872 bytes. Median PEAK/PACK and DP/PACK ratios are 0.301 and 0.206. A final-aggregation variable-shadowing error was repaired post hoc from preserved raw/process records without rerunning measurements. Both campaigns retain every outcome, with no value disagreement, invalid or unstarted unit. Timings are single samples on structural inputs, not an application distribution.

A retained 874-input charged-order exploration compares 7,866 roots at budgets 0..8. All succeed; only the known zero-fee four-job case gains from adaptive order (24 versus 26). None of 840 new independent inputs does, although post hoc analysis finds 16 where the best c -sorted order is suboptimal. These finite observations do not extend Theorem 10 beyond its conditions.

Separate zero-fee evidence has 4,232 price inputs, 19,644 native paths/48 DAGs, and adaptive gains in only 2/192 roots from one input. At $B = 1$, CP-SAT/screened DP/full construction complete 78/68/53 roots; others time out. All costs, failures, invalids and unstarted checks remain in the artifact.

9 Related Work

Synchronization synthesis and fallback. Kung and Robinson [1], Section 3.3/p.220, already address repeated validation failure by retaining the critical-section semaphore during reexecution. Cached fallback is therefore an inherited mechanism; Theorem 3 gives its exact resource boundary for the stated call interface. Černý et al. [18] already choose optimistic retries, guards and caches. Their Theorem 6 preserves limit-average values through matched distributions/weights; an erratum supersedes the original hardness claim. Our potential establishes initial worst-total-cost equality for charged calls and actual writes. Their later work [19], Sections 6.1–6.2, optimizes protected-statement counts and profiles block costs under contention and lock acquisition. Counting protection and synchronization is inherited. Our call-cap frontier is worst- L optimal for every writer budget without receiving it; performance calibration remains open. Weighted reachability already expresses optimal worst fault cost [7]; resilience/cost synthesis and general game solvers are established [9, 10].

Reduction and robust dynamic programming. Gangamreddypalli et al. [20] preserve failures and reach original successful final states or failure. Our calls do not commute unconditionally with same-key writes; the potential gives no exact intermediate-state quotient. A cost-preserving reduction remains unestablished. Mannor et al. [8], Section 4/Theorem 5, provide finite global-deviation-budget DP: unfinished upsets, completing nominal branches and token-consuming failures instantiate it with negated cost and horizon $n + B$. Postek et al. [30] supply exact finite-candidate DP, state expansion and causal budgeted routing. These express the finite game; normal form and all-budget closure need further arguments.

The zero-fee package [34] supplies the reused hardness, floor/cap lemma and two-fee ordering/packing. Its lineage map separates the atomic-call potential, universal resource frontier, second slope, sharp ratio and mixed-price reduction.

Envelopes, reflection and allocation. Burdzy et al. [27], Equations (2.12)–(2.13)/Theorem 2.6, give the general reflected recurrence; our floor/cap lemma specializes it to concavity. Marginal insertion is elementary. Bertsimas and Sim [31], Theorem 3/Equations (15)–(19), already give an input-deviation slope envelope $Z(\Gamma) = \min_t \{\Gamma d_t + F_t\}$. Such envelopes alone are not new. Here Equations (6)–(8) close the operations for repeated causal failures, two completing modes and dependent job

choices. Reduction to a fixed binary robust set remains unestablished. Concave allocation [12, 13] supplies lower bounds; decision-dependent information and controllable uncertainty are broader settings [15, 16].

Scheduling and hardening. At one failure, the zero-fee problem couples processing/tail modes $(p_i, 0)$ and $(0, c_i)$. Fixed modes use Lawler’s precedence sequencing [28]; our hardness concerns selecting modes. Izosimov et al. [21] activate DAG schedules under bounded failure histories with signalling/transparency constraints; An and Yang [11] synthesize stochastic hardening under fixed priorities. We omit those constraints. Chrobak et al. [14], Lemma 4/Theorem 5, concentrate faults on a critical job and compute prescribed-order latest completion in $O(n)$ independently of the fault bound. Their model mandates retry; ours chooses priced protection/cached completion and returns all-budget counted-work policies. Bold and Goerigk [32] use $\Gamma + 1$ DAG layers and a compact MIP after duration observation. Their activities complete despite delay; our failure leaves a job unfinished before another causal choice.

Ordering and rent-or-buy. Heterogeneous thresholds and ordering appear in ski rental [22, 26]. The direct map $f(H) = \sum_{i \in H} p_i$, $g(H) = \sum_{i \in H} c_i$, $T = B$ gives 6 for $(p, c) = (2, 5), (4, 6)$ at $B = 1$, versus our zero-fee value 5. This excludes only that map: shared interference, mandatory work and causal completion must be matched. At $B = 1$, charged fixed-order excess is $\psi_{\pi_1} \circ \dots \circ \psi_{\pi_n}(0)$ with $\psi_i(x) = \min\{x + P_i, \max(x, c_i), \delta_i + \max(x, s_i)\}$. Kawase et al. [3] establish composition ordering and tractable/hard scheduling classes. This inherited scalar view does not directly resolve our nonconvex/nonconcave maps or all-budget closure.

10 Limitations and Threats to Validity

The theorem requires the whole-kernel, captured-parent, persistent-completion interface of Section 2. Partial/incremental work, replenished budgets, structural changes, exceptions and external milestone completion are excluded. A free certificate that all future writers have finished can reduce cost to baseline; the map supplies neither this certificate nor calibrated prices. Scalar minimax values use a supplied B ; Theorem 4 removes that input only for the stated resource objective.

The zero-fee interface permits individual/batch acquisition and retained guards, with two-mode equality. Charged failed-acquisition experiments contain five strict improvements among 384 inputs, including unrestricted 242 versus macro 243. Immediately releasing or completing a failed guard is also refuted at an intermediate state (43 versus 45). Charged exposed-guard equality from all-unprepared states remains unproved despite finite agreement; the atomic-call theorem does not resolve it.

Deephaven guarantees require fixed keys/hierarchy, stated Action changes, owned finite refreshes and synchronized boundaries. Ordinary Web keys are static; periodic updates supply no finite q , and equal clock parity can hide different notification states. Whole-system optimality and prevalence remain unknown; zero prefix aborts and policy-dependent writer cuts are retained.

Finite authored tests and source-model traces do not mechanically verify the proofs. Comparator gains are sparse; direct methods sometimes avoid full-construction timeouts. Calibration establishes neither positive measured premiums nor timing benefit; post-outcome criterion changes cannot repair it. Developer-effort reduction and human decision improvement were not measured.

11 Conclusion

Charged atomic completion yields an exact normal form, a resource frontier attained without a writer-budget input, and all-budget compilation with a compact mixed-price class. A sharp ratio bounds independent-job savings; conditional Java correspondence and snapshot integration delimit native guarantees.

Data Availability

The complete charged-call, resource-frontier, sharp-ratio and Deephaven supplement is public at commit `ec02a7a` [35]. It includes fixed inputs, constructors/checkers, native sources and raw traces, complete outcome ledgers, seventeen standard replay stages, optional Java runs and a theorem-lineage map. The zero-free package [34] remains unchanged. Oversized input ledgers use lossless compression with decoded hashes. Licensed modified Deephaven sources retain their full license and notice; upstream archives, runtimes, build caches, third-party articles and private correspondence are excluded.

References

- [1] H. T. Kung and John T. Robinson. 1981. On optimistic methods for concurrency control. *ACM Transactions on Database Systems* 6, 2, 213–226. <https://doi.org/10.1145/319566.319567>.
- [2] OpenJDK. n.d. *JMH: Java Microbenchmark Harness*, version 1.37. Accessed September 9, 2026. <https://github.com/openjdk/jmh/tree/1.37>.
- [3] Yasushi Kawase, Kazuhisa Makino, and Kento Seimi. 2016. Optimal Composition Ordering Problems for Piecewise Linear Functions. In *ISAAC, LIPIcs* 64, 42:1–42:13. <https://doi.org/10.4230/LIPIcs.ISAAC.2016.42>. Extended version: *Algorithmica* (2018), <https://doi.org/10.1007/s00453-017-0397-y>.
- [4] Deephaven Data Labs. 2026. *Deephaven Core source*, commit 6367313a319b79437d4d2116c77836ad56d5138d. Files `ConstructSnapshot.java`, `HierarchicalTableImpl.java`, `EventDrivenUpdateGraph.java`, and `JsTreeTable.java`. Accessed September 9, 2026. <https://github.com/deephaven/deephaven-core/tree/6367313a319b79437d4d2116c77836ad56d5138d>.
- [5] Deephaven Data Labs. 2026. *tree: Real-time tree table*. Python documentation, version 0.40.7. Accessed September 9, 2026. <https://deephaven.io/core/0.40.7/docs/reference/table-operations/create/tree/>.
- [6] Deephaven Data Labs. 2026. *Execution context*. Official documentation, custom update graphs. Accessed September 9, 2026. <https://deephaven.io/core/docs/conceptual/execution-context/>.
- [7] Emil Dumitrescu, Alain Girault, Hervé Marchand, and Éric Rutten. 2007. *Optimal discrete controller synthesis for the modeling of fault-tolerant distributed systems*. INRIA Research Report RR-6137. https://www.researchgate.net/publication/250079523_Optimal_discrete_controller_synthesis_for_modeling_fault-tolerant_distributed_systems.
- [8] Shie Mannor, Ofir Mebel, and Huan Xu. 2012. Lightning Does Not Strike Twice: Robust MDPs with Coupled Uncertainty. In *ICML*. <https://icml.cc/2012/papers/215.pdf>.
- [9] Daniel Neider, Alexander Weinert, and Martin Zimmermann. 2020. Synthesizing optimally resilient controllers. *Acta Informatica* 57, 195–221. <https://doi.org/10.1007/s00236-019-00345-7>.
- [10] Alexander Weinert. 2019. *Optimality and Resilience in Parity Games*. PhD thesis, Saarland University. Section 6.2. https://finkbeiner.groups.cispa.de/_assets/paper.C89PMq4G.pdf.
- [11] Hyung-Chan An and Hoeseok Yang. 2021. Fully Adaptive Stochastic Handling of Soft-Errors in Real-Time Systems. *IEEE Access* 9, 155058–155071. <https://doi.org/10.1109/ACCESS.2021.3128397>.
- [12] Awi Federgruen and Henri Groenevelt. 1986. The Greedy Procedure for Resource Allocation Problems: Necessary and Sufficient Conditions for Optimality. *Operations Research*. https://business.columbia.edu/sites/default/files-efs/pubfiles/4071/federgruen_greedy_procedure.pdf.
- [13] Dimitris Bertsimas and Melvyn Sim. 2004. The Price of Robustness. *Operations Research* 52, 1, 35–53. <https://www.mit.edu/~dbertsim/papers/Robust%20Optimization/The%20price%20of%20Robustness.pdf>.
- [14] Marek Chrobak, Mathilde Hurand, and Jiří Sgall. 2009. Algorithms for Testing Fault-Tolerance of Sequenced Jobs. *Journal of Scheduling*. Author version. <https://iuuk.mff.cuni.cz/~sgall/ps/faults.pdf>.
- [15] Phebe Vayanos, Angelos Georgioui, and Han Yu. 2026. Robust Optimization with Decision-Dependent Information Discovery. *Management Science* 72, 2, 1509–1528. <https://doi.org/10.1287/mnsc.2021.00160>.
- [16] Eva Ley, Rebecca Marx, Maximilian Merkert, Tim Niemann, and Sebastian Stiller. 2026. Decision-dependent robust optimization with elementwise controllable uncertainty sets. *OR Spectrum*. <https://doi.org/10.1007/s00291-026-00856-z>.
- [17] PRISM-games developers. 2026. PRISM-games. Official tool documentation. Version 3.2.4 used in the experiments. <https://www.prismmodelchecker.org/games/>.
- [18] Pavol Černý, Krishnendu Chatterjee, Thomas A. Henzinger, Arjun Radhakrishna, and Rohit Singh. 2011. Quantitative Synthesis for Concurrent Programs. In *CAV*, 243–259. https://doi.org/10.1007/978-3-642-22110-1_20. Author publication page and complexity erratum: <https://www.microsoft.com/en-us/research/publication/quantitative-synthesis-for-concurrent-programs/>.
- [19] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2015. Optimizing Solution Quality in Synchronization Synthesis. *arXiv preprint arXiv:1511.07163v1*.

- <https://arxiv.org/abs/1511.07163>.
- [20] Namratha Gangamreddypalli, Constantin Enea, and Shaz Qadeer. 2026. Reduction for Structured Concurrent Programs. *arXiv preprint arXiv:2601.13341*. <https://arxiv.org/abs/2601.13341>.
- [21] Viacheslav Izosimov, Paul Pop, Petru Eles, and Zebo Peng. 2012. Scheduling and Optimization of Fault-Tolerant Embedded Systems with Transparency/Performance Trade-Offs. *ACM Transactions on Embedded Computing Systems* 11, 3, Article 61, 35 pages. <https://doi.org/10.1145/2345770.2345773>.
- [22] Bingham Wu, Wei Bao, Dong Yuan, and Bing Zhou. 2022. Competitive Analysis for Multi-Commodity Ski-Rental Problem. In *IJCAI*, 4672–4678. <https://doi.org/10.24963/ijcai.2022/648>.
- [23] Oracle. 2021. ConcurrentHashMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html>.
- [24] Oracle. 2021. ConcurrentMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentMap.html>.
- [25] OpenJDK. 2026. ConcurrentHashMap.java, jdk 17 u source, commit 982d53c227943a511e8d61694feced764b71a6c1. Accessed September 7, 2026. <https://github.com/openjdk/jdk17u/blob/982d53c227943a511e8d61694feced764b71a6c1/src/java.base/share/classes/java/util/concurrent/ConcurrentHashMap.java>.
- [26] Ziwei Li, Bo Sun, Zhiqiu Zhang, Mohammad Hajiesmaili, Bingham Wu, Lin F. Yang, and Yang Gao. 2025. Combinatorial Ski Rental Problem: Robust and Learning-Augmented Algorithms. In *NeurIPS*. https://papers.nips.cc/paper_files/paper/2025/hash/ae3e173398d6e43fea63cbcc16fbaa98-Abstract-Conference.html.
- [27] Krzysztof Burdzy, Weining Kang, and Kavita Ramanan. 2009. The Skorokhod problem in a time-dependent interval. *Stochastic Processes and their Applications* 119, 2, 428–452. <https://doi.org/10.1016/j.spa.2008.03.001>.
- [28] Eugene L. Lawler. 1973. Optimal Sequencing of a Single Machine Subject to Precedence Constraints. *Management Science* 19, 5, 544–546. <https://doi.org/10.1287/mnsc.19.5.544>.
- [29] Laurent Perron, Frédéric Didier, and Steven Gay. 2023. The CP-SAT-LP Solver (Invited Talk). In *CP, LIPIcs* 280, 3:1–3:2. <https://doi.org/10.4230/LIPIcs.CP.2023.3>.
- [30] Krzysztof Postek, Ward Romeijnders, and Wolfram Wiesemann. 2025. Technical Note: Multi-Stage Robust Mixed-Integer Programming. *Operations Research* 73, 6, 3345–3358. <https://doi.org/10.1287/opre.2023.0520>. Online appendix; author code.
- [31] Dimitris Bertsimas and Melvyn Sim. 2003. Robust discrete optimization and network flows. *Mathematical Programming* 98, 49–71. <https://doi.org/10.1007/s10107-003-0396-4>.
- [32] Matthew Bold and Marc Goerigk. 2021. A compact reformulation of the two-stage robust resource-constrained project scheduling problem. *Computers & Operations Research*. <https://doi.org/10.1016/j.cor.2021.105232>. Accepted manuscript.
- [33] Richard M. Karp. 1972. Reducibility Among Combinatorial Problems. In *Complexity of Computer Computations*, 85–103. https://doi.org/10.1007/978-1-4684-2001-2_9. Primary paper.
- [34] Anonymous Authors. 2026. *Compiling Optimal Retry Policies for Dependent Transformations: Research Artifact*. Version 5 with publication supplement, commit e0e0110. <https://github.com/research-artifact-archive/research-artifact/tree/e0e011023c0591bbbf422390ad8d29785ad0023f>.
- [35] Anonymous Authors. 2026. *Compiling Optimal Retry Policies with Charged Atomic Calls: Research Artifact*. Version 9 resource supplement, commit ec02a7a. <https://github.com/research-artifact-archive/research-artifact/tree/ec02a7a14f47413c0441841f39986e736a29d446>.